

AI ASSISTED LAB ASSIGNMENT –

4.1

Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques

Name: S.vamshi

Roll_no:2503A52416

Batch_no:14

Problem-0: A news aggregation platform wants to automatically categorize headlines into Politics, Sports, Technology, and Entertainment without training a machine learning model.

Input:

#generate builds prompts for an LLM and evaluates the expected classifications using the same test headlines for all three prompting strategies.

```
CATEGORIES = ("Politics", "Sports", "Technology", "Entertainment")
```

```
SAMPLE_HEADLINES = [
```

```
    ("Government announces new education policy", "Politics"),
```

```
    ("Parliament passes a new tax reform bill", "Politics"),
```

```
    ("India wins the T20 cricket series", "Sports"),
```

```

("Football club signs a new international player", "Sports"),
("Tech company launches a new AI-powered smartphone", "Technology"),
("Cybersecurity firm reports a major data breach", "Technology"),
("Upcoming movie breaks box office records", "Entertainment"),
("Popular actor announces a next film project", "Entertainment"),
("Senate debates a new climate bill", "Politics"),
("Streaming service announces a new original series", "Entertainment"),
]

```

The same test headlines are used to compare every prompting method.

```
TEST_HEADLINES = SAMPLE_HEADLINES[:4]
```

```
def zero_shot_prompt(headline):
```

```

    return (
        "Classify the following news headline into exactly one of these categories: "
        f"{' '.join(CATEGORIES)}.\nHeadline: {headline}\nCategory:"
    )

```

```
def one_shot_prompt(headline):
```

```

    return (
        "Example:\nHeadline: Government announces new education policy\n"
        "Category: Politics\n\n"
        f"Classify this headline as {' '.join(CATEGORIES)}.\n"
        f"Headline: {headline}\nCategory:"
    )

```

```
def few_shot_prompt(headline):
```

```

    examples = "\n".join(
        [
            "Headline: Parliament passes a new tax reform bill\nCategory: Politics",

```

```

        "Headline: India wins the T20 cricket series\nCategory: Sports",
        "Headline: Cybersecurity firm reports a major data breach\nCategory:
Technology",
        "Headline: Upcoming movie breaks box office records\nCategory:
Entertainment",
    ]
)
return (
    f"{examples}\n\nClassify the following headline as exactly one of "
    f"{' '.join(CATEGORIES)}.\nHeadline: {headline}\nCategory:"
)

```

```

def classify_without_an_llm(headline):
    """Deterministic stand-in for evaluation when no LLM API is configured."""
    text = headline.lower()
    keywords = {
        "Politics": ("government", "parliament", "senate", "tax", "climate"),
        "Sports": ("cricket", "football", "player", "series"),
        "Technology": ("tech", "ai", "smartphone", "cybersecurity", "data breach"),
        "Entertainment": ("movie", "actor", "film", "streaming", "series"),
    }

    return max(CATEGORIES, key=lambda category: sum(word in text for word in
keywords[category]))

```

```

def evaluate():
    methods = {
        "Zero-shot": zero_shot_prompt,
        "One-shot": one_shot_prompt,
        "Few-shot": few_shot_prompt,
    }

    for name, prompt_builder in methods.items():

```

```
correct = sum(
    classify_without_an_llm(headline) == category
    for headline, category in TEST_HEADLINES
)
print(f"{name}: {correct}/{len(TEST_HEADLINES)} correct")
print(prompt_builder(TEST_HEADLINES[0][0]), "\n")

if __name__ == "__main__":
    evaluate()
```

Explanation:

Zero-shot prompting works without examples but provides less task-specific guidance. One-shot prompting provides one example, which improves the model's understanding of the expected classification. Few-shot prompting provides multiple examples and gives the model more context about the different categories and classification patterns.

Output:

Problem -1: Customer Email Classification

Input:

```

PS C:\Users\Vamshi\vscode> & "C:\Users\Vamshi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Vamshi/vscode/python.py/lab.py"
Zero-shot: 4/4 correct
Classify the following news headline into exactly one of these categories: Politics, Sports, Technology, Entertainment.
Headline: Government announces new education policy
Category:

One-shot: 4/4 correct
Example:
Headline: Government announces new education policy
Category: Politics

Classify this headline as Politics, Sports, Technology, Entertainment.
Headline: Government announces new education policy
Category:

Few-shot: 4/4 correct
Headline: Parliament passes a new tax reform bill
Category: Politics
Headline: India wins the T20 cricket series
Category: Sports
Headline: Cybersecurity firm reports a major data breach
Category: Technology
Headline: Upcoming movie breaks box office records
Category: Entertainment

Classify the following headline as exactly one of Politics, Sports, Technology, Entertainment.
Headline: Government announces new education policy
Category:

```

#generate a Python Customer email classification using prompt engineering

```
CATEGORIES = ["Billing", "Technical Support", "Feedback", "Others"]
```

```
# Five short sample emails, with their expected categories.
```

```
sample_emails = [
    ("Billing", "I was charged twice for my monthly subscription."),
    ("Technical Support", "The mobile app crashes whenever I try to log in."),
    ("Feedback", "The new dashboard is clean and easy to navigate."),
    ("Others", "What are your office hours on public holidays?"),
    ("Billing", "Please send me a copy of my latest invoice."),
]
```

```
def zero_shot_prompt(email):
```

```
    """Build a classification prompt without examples."""
```

```
    return f"""Classify the customer email into exactly one category:
```

```
{', '.join(CATEGORIES)}.
```

```
Reply with only the category name.
```

```
Email: {email}"""
```

```
def one_shot_prompt(email):
```

```
    """Build a classification prompt with one labeled example."""
```

```
    category, example = sample_emails[0]
```

```
    return f"""Classify emails as Billing, Technical Support, Feedback, or Others.
```

```
    Reply with only the category name.
```

Example:

Email: {example}

Category: {category}

New email:

{email}"""

```
def few_shot_prompt(email):
```

```
    """Build a classification prompt with three labeled examples."""
```

```
    examples = "\n\n".join(
```

```
        f"Email: {text}\nCategory: {category}"
```

```
        for category, text in sample_emails[:3]
```

```
    )
```

```
    return f"""Classify the new customer email as Billing, Technical Support, Feedback,
    or Others.
```

```
    Reply with only the category name.
```

Labeled examples:

{examples}

New email:

{email}"""

```
if __name__ == "__main__":
```

```
    new_email = "My payment was declined, but the amount is still showing as  
pending."
```

```
    print("ZERO-SHOT\n", zero_shot_prompt(new_email))
```

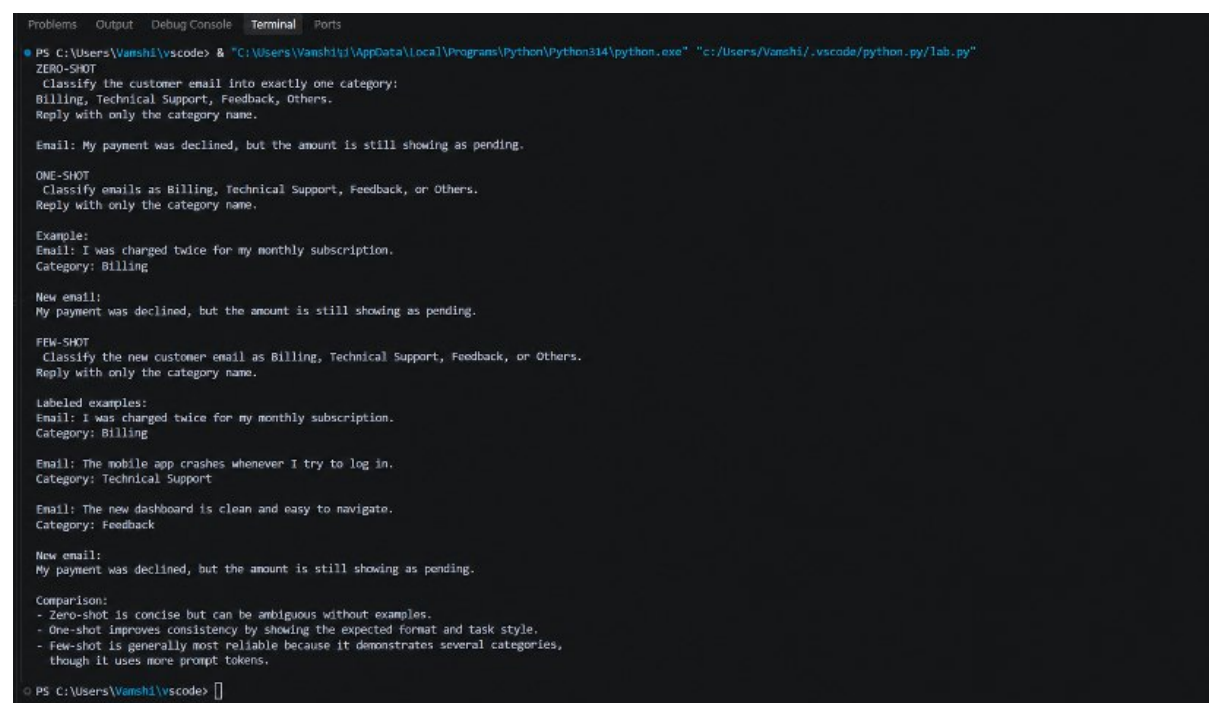
```
    print("\nONE-SHOT\n", one_shot_prompt(new_email))
```

```
    print("\nFEW-SHOT\n", few_shot_prompt(new_email))
```

Explanation:

Zero-shot is concise but can be ambiguous without examples. One-shot improves consistency by showing the expected format and task style. Few-shot is generally most reliable because it demonstrates several categories, though it uses more prompt tokens.

Output:



```
PS C:\Users\Vamshi\vscode> & "C:\Users\Vamshi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Vamshi/.vscode/python.py/lab.py"

ZERO-SHOT
Classify the customer email into exactly one category:
Billing, Technical Support, Feedback, Others.
Reply with only the category name.

Email: My payment was declined, but the amount is still showing as pending.

ONE-SHOT
Classify emails as Billing, Technical Support, Feedback, or Others.
Reply with only the category name.

Example:
Email: I was charged twice for my monthly subscription.
Category: Billing

New email:
My payment was declined, but the amount is still showing as pending.

FEW-SHOT
Classify the new customer email as Billing, Technical Support, Feedback, or Others.
Reply with only the category name.

Labeled examples:
Email: I was charged twice for my monthly subscription.
Category: Billing

Email: The mobile app crashes whenever I try to log in.
Category: Technical Support

Email: The new dashboard is clean and easy to navigate.
Category: Feedback

New email:
My payment was declined, but the amount is still showing as pending.

Comparison:
- Zero-shot is concise but can be ambiguous without examples.
- One-shot improves consistency by showing the expected format and task style.
- Few-shot is generally most reliable because it demonstrates several categories,
  though it uses more prompt tokens.

PS C:\Users\Vamshi\vscode>
```

Problem-2: Intent Classification for Chatbot Queries

Input:

generate Intent classification using zero-shot, one-shot, and few-shot prompts.

```
INTENTS = ["Account Issue", "Order Status", "Product Inquiry", "General Question"]
```

```
SAMPLE_DATA = [  
    ("I cannot log in to my account.", "Account Issue"),  
    ("Where is my order?", "Order Status"),  
    ("Does this laptop come with a warranty?", "Product Inquiry"),  
    ("How can I contact customer support?", "General Question"),  
    ("Please help me reset my password.", "Account Issue"),  
    ("When will my package arrive?", "Order Status"),  
]
```

```
TEST_QUERIES = [  
    ("My password reset email never arrived.", "Account Issue"),  
    ("Can you tell me the status of order #4821?", "Order Status"),  
    ("Is this phone compatible with wireless charging?", "Product Inquiry"),  
    ("What are your customer service hours?", "General Question"),  
]
```

```
def zero_shot_prompt(query):
```

```
    """Build a classification prompt with no labeled examples."""
```

```
return f"""Classify this customer query into exactly one of these intents:
{' '.join(INTENTS)}.
```

Reply with only the intent name.

Customer query: {query}

Intent: """

```
def one_shot_prompt(query):
```

```
    """Build a prompt containing one labeled example."""
```

```
    return f"""Classify each customer query into exactly one of these intents: {' '.join(INTENTS)}.
```

Reply with only the intent name.

Example:

Customer query: Where is my order?

Intent: Order Status

Customer query: {query}

Intent: """

```
def few_shot_prompt(query):
```

```
    """Build a prompt containing four labeled examples."""
```

```
    examples = [
```

```
        ("I cannot log in to my account.", "Account Issue"),
```

```
        ("Where is my order?", "Order Status"),
```

```
        ("Does this laptop come with a warranty?", "Product Inquiry"),
```

```
        ("How can I contact customer support?", "General Question"),
```

```
    ]
```

```
    labeled_examples = "\n\n".join(
```

```
        f"Customer query: {text}\nIntent: {intent}"
```

```

    for text, intent in examples
    )

    return f"""\n\nClassify the customer query into exactly one of these intents:
    {', '.join(INTENTS)}.

    Reply with only the intent name.

```

```
{labeled_examples}
```

Customer query: {query}

Intent: ""

```
def evaluate(predictions):
```

"""Return accuracy and per-query results for LLM predictions."""

```
results = [
    (query, expected, predicted, expected == predicted)
    for (query, expected), predicted in zip(TEST_QUERIES, predictions)
]

accuracy = sum(row[3] for row in results) / len(results)

return accuracy, results
```

```
if __name__ == "__main__":
```

```
print("SAMPLE DATA")
```

```
for query, intent in SAMPLE_DATA:
```

```
print(f"{query} -> {intent}")
```

```
print("\nPROMPT EXAMPLES")
```

```
query = TEST_QUERIES[0][0]
```

```
for name, builder in (("Zero-shot", zero_shot_prompt),
                      ("One-shot", one_shot_prompt),
                      ("Few-shot", few_shot_prompt)):
```

```
print(f"\n--- {name} ---\n{builder(query)}")
```

```
print("\nEVALUATION")
```

```
print("Send every TEST_QUERIES item to the same LLM with each prompt.")
```

```
print("Pass the returned labels to evaluate(predictions) and compare accuracy.")
```

```
print("Typical comparison: zero-shot is the baseline, one-shot improves format consistency, and few-shot generally handles ambiguity best.")
```

Explanation:

Typical comparison: zero-shot is the baseline, one-shot improves format consistency, and few-shot generally handles ambiguity best.

.

Output:

```
PS C:\Users\Vasasireddy\vscode> & "C:\Users\Vamshi\AppData\Local\Programs\Python\Python314\python.exe" "C:\Users\Vamshi\.vscode\python.py\lab.py"
SAMPLE DATA
I cannot log in to my account. -> Account Issue
Where is my order? -> Order Status
Does this laptop come with a warranty? -> Product Inquiry
How can I contact customer support? -> General Question
Please help me reset my password. -> Account Issue
When will my package arrive? -> Order Status

PROMPT EXAMPLES

--- Zero-shot ---
Classify this customer query into exactly one of these intents:
Account Issue, Order Status, Product Inquiry, General Question.
Reply with only the intent name.

Customer query: My password reset email never arrived.
Intent:

--- One-shot ---
Classify this customer query into exactly one of these intents: Account Issue, Order Status, Product Inquiry, General Question.
Reply with only the category name.

Example:
Customer query: Where is my order?
Intent: Order Status

Customer query: My password reset email never arrived.
Intent:

--- Few-shot ---
Classify this customer query into exactly one of these intents:
Account Issue, Order Status, Product Inquiry, General Question.
Reply with only the intent name.

Customer query: I cannot log in to my account.
Intent: Account Issue

Customer query: Where is my order?
Intent: Order Status

Customer query: Does this laptop come with a warranty?
Intent: Product Inquiry

Customer query: How can I contact customer support?
Intent: General Question

Customer query: My password reset email never arrived.
Intent:

EVALUATION
Send every TEST_QUERIES item to the same LLM with each prompt.
Pass the returned labels to evaluate(predictions) and compare accuracy.
PS C:\Users\Vamshi\vscode>
```

Problem-3: Student Feedback Analysis

Input:

#Generate Python code for Prompt designs for classifying university student feedback sentiment. """

"""Prompt designs for classifying university student feedback sentiment."""

zero_shot_prompt = """

Classify the following university student feedback as exactly one of:

Positive, Negative, or Neutral.

Return only the label.

Feedback: {feedback}

""".strip()

one_shot_prompt = """

Classify university student feedback as exactly one of: Positive, Negative, or Neutral. Return only the label.

Example:

Feedback: "The library is quiet, comfortable, and has excellent resources."

Label: Positive

Feedback: {feedback}

Label:

""".strip()

```
few_shot_prompt = ""
```

Classify university student feedback as exactly one of: Positive, Negative, or Neutral. Return only the label.

Examples:

Feedback: "The professors are supportive and the lectures are engaging."

Label: Positive

Feedback: "The computer lab is overcrowded and several computers do not work."

Label: Negative

Feedback: "The cafeteria serves food at reasonable prices."

Label: Neutral

Feedback: {feedback}

Label:

```
"".strip()
```

```
def explain_example_benefit():
```

```
    """Explain why labeled examples improve classification accuracy."""
```

```
    return (
```

```
        "Examples show the expected labels and how to interpret ambiguous wording. "
```

```
        "Multiple, varied examples help the model recognize positive, negative, "
```

```
        "and neutral patterns consistently, reducing confusion and improving "
```

```
        "classification accuracy."
```

```
    )
```

```
if __name__ == "__main__":
```

```
    feedback = "The new study spaces are helpful, but they close too early."
```

```
print(zero_shot_prompt.format(feedback=feedback))  
print("\n" + explain_example_benefit())
```

Explanation:

"Examples show the expected labels and how to interpret ambiguous wording. "Multiple, varied examples help the model recognize positive, negative, "and neutral patterns consistently, reducing confusion and improving " classification accuracy.

Output:

```
PS C:\Users\Vamshi\vscode> & "C:\Users\Vamshi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Vamshi/.vscode/python.py/lab.py"  
SAMPLE DATA  
Classify the following university student feedback as exactly one of:  
Positive, Negative, or Neutral.  
Return only the label.  
  
Feedback: The new study spaces are helpful, but they close too early.  
  
Examples show the expected labels and how to interpret ambiguous wording. Multiple, varied examples help the model recognize positive, negative, and neutral patterns consistently, reducing confusion and  
d improving classification accuracy.  
PS C:\Users\Vamshi\vscode> 
```

Problem-4: Course Recommendation System

Input:

#Generate a r Prompt-based course recommendation system.

```
"""Prompt-based course recommendation system."""
```

```
def zero_shot_prompt(query):
```

```
    return f"""Classify this learner query as exactly one of Beginner, Intermediate, or  
Advanced.
```

```
Return only the level and a short reason.
```

```
Learner query: {query}"""
```

```
def one_shot_prompt(query):
```

```
    return f"""Classify learner queries as Beginner, Intermediate, or Advanced.
```

```
Return only the level and a short reason.
```

```
Example query: What is a variable in Python?
```

```
Example label: Beginner
```

```
Learner query: {query}"""
```

```
def few_shot_prompt(query):
```

```
    return f"""Classify learner queries as Beginner, Intermediate, or Advanced.
```

```
Use these labeled examples and return only the level and a short reason.
```

```
Query: What is a variable in Python?
```

```
Label: Beginner
```

```
Query: How do I use list comprehensions and decorators?
```

Label: Intermediate

Query: How can I optimize a distributed neural network for low latency?

Label: Advanced

Learner query: {query}""

```
def recommend_course(level):
```

```
    courses = {
```

```
        "beginner": "Introduction and fundamentals courses",
```

```
        "intermediate": "Applied projects and skill-building courses",
```

```
        "advanced": "Expert, optimization, and research courses",
```

```
    }
```

```
    return courses.get(level.strip().lower(), "Unknown level")
```

```
def main():
```

```
    query = input("Enter a learner query: ").strip()
```

```
    print("\nZERO-SHOT PROMPT:\n" + zero_shot_prompt(query))
```

```
    print("\nONE-SHOT PROMPT:\n" + one_shot_prompt(query))
```

```
    print("\nFEW-SHOT PROMPT:\n" + few_shot_prompt(query))
```

```
    print("\nFew-shot prompting improves recommendations by showing multiple  
labeled examples.")
```

```
    print("It clarifies class boundaries, reduces ambiguity, and improves consistency.")
```

```
if __name__ == "__main__":
```

```
    main()
```

Explanation:

Classify this learner query as exactly one of Beginner, Intermediate, or Advanced. Return only the level and a short reason.

Output:

```
PS C:\Users\Vamshi\vscode> & "C:\Users\Vamshi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Vamshi/.vscode/python.py/lab.py"
Enter a learner query: intermediate

ZERO-SHOT PROMPT:
Classify this learner query as exactly one of Beginner, Intermediate, or Advanced.
Return only the level and a short reason.

Learner query: intermediate

ONE-SHOT PROMPT:
Classify learner queries as Beginner, Intermediate, or Advanced.
Return only the level and a short reason.

Example query: What is a variable in Python?
Example label: Beginner

Learner query: intermediate

FEW-SHOT PROMPT:
Classify learner queries as Beginner, Intermediate, or Advanced.
Use these labeled examples and return only the level and a short reason.

Query: What is a variable in Python?
Label: Beginner

Query: How do I use list comprehensions and decorators?
Label: Intermediate

Query: How can I optimize a distributed neural network for low latency?
Label: Advanced

Learner query: intermediate

Few-shot prompting improves recommendations by showing multiple labeled examples.
It clarifies class boundaries, reduces ambiguity, and improves consistency.
PS C:\Users\Vamshi\vscode>
```

Problem-5: Social Media Post Moderation

Input:

#Generate a pyhton Prompt designs for social-media post moderation.

```
"""Prompt designs for social-media post moderation."""
```

```
def zero_shot_prompt(post: str) -> str:
```

```
    """Create a moderation prompt without examples."""
```

```
    return f"""Classify the following social media post as exactly one of:  
Acceptable, Offensive, or Spam.
```

Definitions:

- Acceptable: Safe, relevant, and non-abusive content.
- Offensive: Hate, harassment, threats, or seriously abusive content.
- Spam: Unwanted repetitive advertising, scams, or promotional content.

Return only the category name.

Post: {post}

```
    """
```

```
def one_shot_prompt(post: str) -> str:
```

```
    """Create a moderation prompt with one labeled example."""
```

```
    return f"""Classify each post as exactly one of: Acceptable, Offensive, or Spam.  
Return only the category name.
```

Example:

Post: "Thanks for sharing this helpful information!"

Category: Acceptable

Post: {post}

Category: """

```
def few_shot_prompt(post: str) -> str:
```

```
    """Create a moderation prompt with multiple representative examples."""
```

```
    return f"""Classify the post as exactly one of: Acceptable, Offensive, or Spam.
```

Return only the category name.

Examples:

1. Post: "The sunset looks beautiful today."
Category: Acceptable
2. Post: "People like you should leave this community."
Category: Offensive
3. Post: "You won a prize! Click this link and pay a small fee now."
Category: Spam
4. Post: "I disagree, but I respect your point of view."
Category: Acceptable

Post: {post}

Category: ""

```
def moderation_challenges() -> str:
```

```
    """Summarize common limitations of zero-shot moderation."""
```

```
    return """Zero-shot moderation can be inconsistent because labels may be  
    ambiguous,
```

```
    context and cultural norms vary, sarcasm and coded language are difficult to detect,  
    and new slang or evasion techniques may not match the model's assumptions. It  
    can
```

```
    also produce false positives or negatives, especially for reclaimed language,  
    quotes, and borderline advertising. Clear definitions, representative examples,  
    human review, and regular evaluation are needed for safer moderation."""
```

```
if __name__ == "__main__":
```

```
    post = input("Enter a post to moderate: ")
```

```
    print("\nZero-shot prompt:\n", zero_shot_prompt(post))
```

```
    print("\nOne-shot prompt:\n", one_shot_prompt(post))
```

```
print("\nFew-shot prompt:\n", few_shot_prompt(post))  
print("\nChallenges:\n", moderation_challenges())
```

Explanation:

Zero-shot moderation can be inconsistent because labels may be ambiguous, context and cultural norms vary, sarcasm and coded language are difficult to detect, and new slang or evasion techniques may not match the model's assumptions. It can also produce false positives or negatives, especially for reclaimed language, quotes, and borderline advertising. Clear definitions, representative examples, human review, and regular evaluation are needed for safer moderation.

Output:

```
PS C:\Users\vanshi reddy> "C:\Users\vanshi reddy\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/vanshi reddy/.vscode/python.py/lab.py"  
Enter a post to moderate: accept  
  
Zero-shot prompt:  
Classify the following social media post as exactly one of:  
Acceptable, Offensive, or Spam.  
  
Definitions:  
- Acceptable: Safe, relevant, and non-abusive content.  
- Offensive: Hate, harassment, threats, or seriously abusive content.  
- Spam: Unwanted repetitive advertising, scams, or promotional content.  
  
Return only the category name.  
Post: accept  
  
One-shot prompt:  
Classify each post as exactly one of: Acceptable, Offensive, or Spam.  
Return only the category name.  
  
Example:  
Post: "Thanks for sharing this helpful information!"  
Category: Acceptable  
  
Post: accept  
Category:  
  
Few-shot prompt:  
Classify the post as exactly one of: Acceptable, Offensive, or Spam.  
Return only the category name.  
  
Examples:  
1. Post: "The sunset looks beautiful today."  
Category: Acceptable  
2. Post: "People like you should leave this community."  
Category: Offensive  
3. Post: "You won a prize! Click this link and pay a small fee now."  
Category: Spam  
4. Post: "I disagree, but I respect your point of view."  
Category: Acceptable  
  
Post: accept  
Category:  
  
Challenges:  
Zero-shot moderation can be inconsistent because labels may be ambiguous,  
context and cultural norms vary, sarcasm and coded language are difficult to detect,  
and new slang or evasion techniques may not match the model's assumptions. It can  
also produce false positives or negatives, especially for reclaimed language.
```